P2889R0
Distributed Arrays

Published Proposal, 2023-05-15

Author:
Lauri Vasama

Audience:
EWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current draft:
vasama.github.io/wg21/D2889.html

Current draft source:
github.com/vasama/wg21/blob/main/D2889.bs

Abstract

A new method of defining global arrays by merging definitions from multiple translation units.

Table of Contents

§ 1. Introduction

It is often desirable to define sets of static objects in a distributed manner across multiple translation units without explicitly and manually maintaining the full list of elements in any single location. This improves software maintainability when the entire set must ultimately be available at runtime.

Is this really worth the committee's time? I believe so. While the feature may seem novel, it is arguably standardising existing practice that is being widely used out in the field. In fact the facility being proposed already is already in use in all major C++ implementations as an implementation strategy for static initialisers, thread local storage and exception handling tables. This paper proposes to expose that existing facility to the programmer.

This feature was previously introduced in [P2268R0] under the name "link-time arrays" with hopes of future standardisation.

§ 1.1. Comparison to state of the art

Here is a quick comparison to static initialiser side effects, which seems to be the solution most often employed by users out in the field today:

Static intialisers	Distributed array

`std::vector<int>& get_array_storage() {`
`  static std::vector<T> vector;`
`  return vector;`
`}`
 	
`std::span<int const> get_array() {`
`  return get_array_storage();`
`}`
 	
`// In another translation unit.`
`[[maybed_unused]]`
`static char const elements_pusher = (`
`  get_array_storage().push_back(value_0),`
`  get_array_storage().push_back(value_1),`
`  0 /* elements_pusher value. */);`
	`extern int const array[];`
	
	`std::span<int const> get_array() {`
	`  return std::span(`
	`  array,`
	`  std::distributed_array_size(array));`
	`}`
	
	`// In another translation unit.`
	`__distributed_array(array)`
	`static int const elements[] = {`
	`  value_0,`
	`  value_1,`
	`};`

2. Existing practice

2.1. Examples of existing usage in C and C++

- Test cases in all major C++ unit test frameworks (Catch2 on GitHub)
- Optimisation passes in LLVM (GitHub)
- Type information in Unreal Engine (Source available on GitHub through Epic Games)
- Type information in Boost.Serialization (GitHub)
- Multiple types of objects in Qt Framework (GitHub)
- Kernels and operations in TensorFlow (GitHub)
- Generators in Apache Thrift (GitHub)
- Initialisation callbacks in the Linux kernel (C language, freestanding) (GitHub)
- Multiple tables defined using linker sections in iPXE (C language, freestanding) (GitHub)
- The author is aware of more usage in proprietary code bases.

2.2. Prior art in other languages

2.2.1. github.com/dtolnay/linkme

Linkme is a Rust library implementing distributed arrays using linker sections. From the readme:

> A distributed slice is a collection of static elements that are gathered into a contiguous section of the binary by the linker. Slice elements may be defined individually from anywhere in the dependency graph of the final binary. The implementation is based on link_section attributes and platform-specific linker support. It does not involve life-before-main or any other runtime initialization on any platform. This is a zero-cost safe abstraction that operates entirely during compilation and linking.

2.2.2. Runtime reflection

In languages such as Java and C# with support for runtime reflection, similar results are achieved by reflecting over the program at runtime and picking out types or functions decorated with specific attributes. In these languages the technique seems especially popular for service discovery paired with dependency injection.

In the case of C++, collation of type information for the purposes of runtime reflection may itself become an important use case in the future if static reflection is added to the language. A runtime reflection library may register descriptions of user types and other entities in distributed arrays for runtime use. This is already being done out in the field to some extent as seen in Unreal Engine and Boost.Serialization.

3. Alternatives considered

3.1. Static initialiser side effects

In standard C++ the only way to achieve the desired result is by building the set at runtime using static initialisers. This approach is widely utilised, but comes with many problems:

- Accessing specific elements adds additional complexity. For example by storing indices in global variables during static initialisation.
- Is subject to the static initialisation order fiasco if other static initialisers should access the set under construction, even when the actual data is known statically and could in theory have been constant initialised.
- Use in freestanding software poses challenges: Dynamic allocation can only be avoided using linked lists or other non-contiguous data structures.
- Is made fragile due to the fact that static initialisers, even when they have side effects, are not always executed if the objects they initialise are never used. In practice this happens when a static library containing translation units with such static initialisers is linked into the program. Any such translation unit containing no externally used definitions is discarded in its entirety. See [basic.start.dynamic] 6.9.3.3.5.
- Conversely static initialisers with side effects cannot be elided when their effects, i.e. the set they build at runtime, are not actually used.
- Adds unnecessary overhead in both time and memory at either initialisation due to dynamic allocation and data copying or during data access due to non-contiguous storage.
- Impedes optimisation as the data is always stored in writable memory even when its mutation after static initialisation is not necessary.

§ 3.2. Linker sections

Outside of standard C++ there exist implementation specific methods to achieve the desired result using linker sections. The variables making up the array elements are all placed into a single linker section using implementation specific pragmas, attributes or linker scripts. The data is accessed at runtime using special symbols placed by the linker at the beginning and end of the section. This is also how static initialisers themselves are generally implemented. The problems with relying on non-portable linker based solutions should be obvious. Apart from the non-portability, this solution also offers poor type safety as variables of differing types may be placed into the section.

§ 4. Design considerations

§ 4.1. Declaring distributed arrays

The current revision of this proposal works by providing definitions for unbounded array declarations (`extern T array[];`) through one or more distributed array definitions. This approach was chosen primarily to minimise impact on the existing standard. However introducing a new kind of declaration for distributed arrays is a viable alternative. The strawman syntax `extern T array[register]` was used in [P2268R0] for such declarations.

§ 4.2. Distributed array type

A new kind of array declaration would be most powerful if combined with a new kind of array type (strawman syntax `T[register]`, `T(&)[register]`, `T(*)[register]`, etc.). This presents a larger change to the language, but comes with some benefits:

- Better and earlier diagnostics due to more clearly stated programmer intent.
- Range-based for loops can be applied to distributed arrays directly.
- `std::ranges` utilities and algorithms can be applied to distributed arrays directly.

Such new distributed array types would behave similarly to the existing unbounded array types, with the addition of a mechanism for querying the size of a distributed array object (see § 4.5 Accessing the size of the distributed array).

§ 4.3. Defining elements of distributed arrays

This proposal uses the strawman syntax `__distributed_array(A)` to specify that a variable definition should also act as a partial definition for a previously declared array `A`. All such partial definitions are merged to form a definition for the array `A`. This definition would conflict with any other definition of `A` which is not a distributed array definition.

Given the declaration `extern cv T A[];`, `__distributed_array(A)` may be applied to any namespace scope variable or static member variable declaration `D` with static storage duration, provided that `D` declares a variable of type `cv T` or an array with element type `cv T`. If `A` is declared `constinit`, `D` must be declared `constinit`. If `A` is declared `const`, `D` may be declared `constexpr`.

It is notable that this proposal makes it possible for the first time in standard C++ to access a global object through two different symbols. There are some use cases for this: consider a graph of initialisation callbacks where nodes refer to

other nodes representing dependencies. Those use cases can generally be solved in other ways using additional indirection. The more important reason for this choice however, is distributed array element templates. In order to contribute to a distributed array through a template, a name is naturally required for that template. Additionally this mechanism maps very directly to how linkers actually work in practice. Existing implementations do currently assume that two distinct global variables declared `extern int a;` and `extern int b[];` do not alias one another. This is problematic and may be another reason to introduce new distributed array declarations (see § 4.1 Declaring distributed arrays).

Examples:

```
extern int const A[];

__distributed_array(A)
static int const E1 = 42;

__distributed_array(A)
constexpr int E2[] = { 1, 2 };

// The template E3 can be used to introduce multiple elements into A.
template<typename T>
__distributed_array(A)
constexpr int E3 = T::some_value;
```

In existing usage this pattern is often known as "registration" with types such as `thing_registry` or `thing_registrar` being commonly used alongside macros such as `REGISTER_THING`. For this reason repurposing the now defunct `register` keyword in place of `__distributed_array` might be a good fit.

§ 4.4. Empty distributed arrays

More so than in the case of regular arrays, it often makes sense to define and use empty distributed arrays. For example, whether the program should perform any initialisation steps provided through a distributed array may depend on its build-time configuration. If empty distributed arrays were not allowed, it would be difficult to detect and handle the case where no initialisation was needed.

If distributed arrays were given new declaration syntax (see § 4.1 Declaring distributed arrays), then a declaration such as `T array[register];` (without `extern`) could act as an empty default definition for the distributed array.

§ 4.5. Accessing the size of the distributed array

There are a number of alternative approaches for exposing the size of the distributed array, each with its own pros and cons.

4.5.1. `sizeof` operator

[P2268R0] put forth the option of defining `sizeof` on distributed arrays as a non-constant-expression. This represents a clear departure from the existing behaviour of the `sizeof` operator. Additionally the size in bytes of the array is generally less useful than the number of elements in the array. Therefore this proposal does not recommend the use of the `sizeof` operator.

4.5.2. Function-like operator

An operator (strawman syntax `__distributed_array_size(A)`) provides the greatest implementation flexibility and safety, but the least user flexibility. The implementation is the least limited in its choice of implementation strategy, and programs using the operator improperly can be made ill formed.

Some alternative syntaxes which avoid the introduction of new keywords include `sizeof register(A)` and `operator register(A)`.

4.5.3. Library template

A function template with a const array reference non-type template parameter can likely provide implementation flexibility and safety equal or near equal to that of an operator, but with potentially slightly greater user flexibility. The two may end up being equal in both flexibility and safety however, in which case the library template might be the preferred option.

Possible implementation:

```
template<auto const& Array>
size_t distributed_array_size() noexcept {
  // The size might be calculated by subtraction of
  // pointers to begin and end symbols generated by the linker.
  return
    __builtin_distributed_array_end(Array) -
    __builtin_distributed_array_begin(Array);
}
```

4.5.4. Library function

A function template with a pointer parameter can provide the greatest user flexibility at the cost of lesser implementation flexibility and safety. Calling this function with a pointer pointing to something other than the first element of a distributed array would have to have undefined behaviour.

Possible implementation:

```
size_t distributed_array_size(auto const* const array) noexcept {
  // The size might be stored before the first element in memory.
  // Note that this otherwise invalid cast exists only
  // to illustrate a potential implementation strategy.
  return reinterpret_cast<size_t const*>(array)[-1];
}
```

If distributed arrays were given distinct types (see § 4.2 Distributed array type) the signature of this function could be changed to `size_t distributed_array_size(auto const(&array)[register])` and incorrect usage could be made ill-formed instead of producing undefined behaviour at runtime.

§ 4.6. Ordering of distributed array elements

- Non-inline elements of a distributed array defined within a single translation unit are ordered according to the order of their definitions.
- Inline elements of a distributed array are ordered relative to other elements according to the order of their definitions, if that order is the same in all translation units. Otherwise the order is unspecified.
- The relative ordering of sets of elements from different translation units is unspecified, but is the same for all distributed arrays. In particular this facilitates the definition of parallel distributed arrays with the same order of elements.

§ 4.7. Non-trivial element types

Accessing a distributed array forces initialisation of variables of static storage duration from all translation units contributing to the array. Order of initialisation of distributed array elements may be left entirely unspecified, except where relative ordering of the elements is specified. However specifying the order of initialisation to match the ordering of the elements should not impose any additional difficulty in implementation of the feature either, as the relative ordering of elements from different translation units is itself unspecified. The initialisation of distributed array elements may be interleaved with initialisation of other non-trivial objects with static storage duration.

§ 4.8. Thread storage duration

Distributed arrays with thread storage duration may have some interesting use cases, but they are not currently being proposed. Any distributed array definition declared `thread_local` is therefore ill-formed.

§ 4.9. Dynamic linking

Any attempt to dynamically link together multiple distributed array definitions into a single array is obviously problematic in the presence of dynamic loading and unloading of libraries. On Windows where each DLL (Dynamic Link Library) is practically its own program, this is not a problem.

On Linux and similar systems the best solution might be for the dynamic linker to consider distributed array definitions for the same array declaration from two shared libraries to be in conflict with one another. In other words, the set of distributed array definitions present when statically linking the shared library would produce a single regular array definition. The dynamic linker would never have to deal with any distributed array definitions. This matches the existing behaviour of linkers for those arrays already being used to implement other features (see § 5 Implementation experience).

With dynamic linking being out of scope for the C++ standard, this issue should not have a major effect on the proposal.

§ 5. Implementation experience

Existing linkers already support merging symbols into arrays from multiple translation units in order to implement static initialisers, thread local storage and exception handling tables. More work is needed to provide a prototype implementation or testimonials from implementers regarding the feasibility of implementing of this feature however.

§ 6. Examples

These examples use the library function syntax to access the size of the distributed array. See § 4.5.4 Library function.

§ 6.1. Unit test framework

The example code is simplified for readability. A real unit test framework would likely use macros to hide any boilerplate code.

§ 6.1.1. Framework header

```
using test_case_callback = void();

// Array of test case callbacks.
extern test_case_callback* const test_cases[];
```

§ 6.1.2. User test code

```
__distributed_array(test_cases)
test_case_callback* const my_test_case = my_test_case_function_1;

__distributed_array(test_cases)
test_case_callback* const my_test_case_array[] = {
  my_test_case_function_2,
  my_test_case_function_3,
};
```

6.1.3. Framework source

```
void run_unit_tests() {
  test_case_callback* const* const test_data = ::test_cases;
  size_t const test_count = std::distributed_array_size(test_data);
  std::span const tests = std::span(test_data, test_count);

  for (test_case_callback* const test : tests) {
    test();
  }
}
```

References

Informative References

[P2268R0]
Ben Craig. *Freestanding Roadmap*. 10 December 2020. URL: https://wg21.link/p2268r0